Conception et Exploitation d'une BDD

Partie 2 • Les Procédures Stockées sous SQL Server

Objectifs du Module

À l'issue de ce cours, vous serez en mesure de maîtriser les concepts avancés de programmation intégrée au SGBD :

- Comprendre le rôle et l'intérêt d'une procédure stockée.
- Créer, modifier, exécuter et supprimer des procédures.
- Gérer la transmission, le typage et la validation des paramètres d'entrée.
- Exploiter les différents mécanismes de retour de résultats.
- Concevoir des traitements métiers robustes intégrant la gestion des transactions et des erreurs.

POSTULAT DE DÉPART : POURQUOI UTILISER DES PROCÉDURES ?

Dans une architecture classique, les requêtes sont souvent individuelles et éparpillées dans le code applicatif. Or, une opération métier (comme la confirmation d'une réservation) nécessite généralement l'exécution cohérente d'un ensemble de requêtes successives. L'objectif de la procédure stockée est de regrouper et d'encapsuler ces traitements pour garantir leur réutilisation.

Intérêts Principaux

- **Éviter la duplication :** Sans procédure, la logique métier risque d'être dupliquée entre différentes interfaces (application mobile, site web). La centralisation facilite grandement la maintenance.
- **Encapsulation :** L'application appelante n'a pas besoin de connaître la structure interne de la base ni les détails du traitement. Elle se contente d'appeler la procédure.
- **Sécurité :** Les procédures offrent un point d'accès étroitement contrôlé. Au lieu d'accorder des droits de modification directs sur les tables, on autorise uniquement l'exécution de la procédure.

Création et Exécution

STRUCTURE D'UNE PROCÉDURE

Une procédure stockée est un véritable programme hébergé dans la base de données. Elle est définie par un nom, d'éventuels paramètres d'entrée ou de sortie, et un bloc principal contenant une ou plusieurs instructions opérationnelles (sélections, mises à jour, contrôles logiques).

Attention : La simple création d'une procédure ne déclenche pas son exécution. Elle est seulement compilée et stockée. Son exécution nécessite un appel explicite de la part du client ou de l'application.

BONNES PRATIQUES DE DÉVELOPPEMENT

- **Mise à jour fluide :** Privilégier les instructions de création combinées à l'altération. Cela évite de devoir supprimer l'objet en cas de modification, ce qui préserve les droits de sécurité qui lui sont associés.
- **Nettoyage des retours :** Il est fortement recommandé de désactiver le renvoi automatique du nombre de lignes affectées au début du traitement. Cela réduit le trafic réseau inutile et prévient certaines erreurs d'interprétation côté client.
- **Conventions de nommage :** Le nom doit immédiatement refléter l'action métier. Une convention courante consiste à associer un verbe d'action à un objet (par exemple : *ListerProduits*, *CreerCommande*). Sont à proscrire les noms génériques ou dénués de sens métier.

GESTION DU CYCLE DE VIE

Les procédures sont gérées via les outils d'administration du SGBD (comme SQL Server Management Studio), au sein des répertoires de programmabilité. Elles peuvent y être consultées, modifiées ou supprimées. Gardez à l'esprit que la suppression d'une procédure est une action définitive qui détruit l'objet de la base.

Paramètres et Validation

RENDRE LE TRAITEMENT DYNAMIQUE

Les paramètres transforment un traitement statique en un algorithme réutilisable et personnalisable. Un paramètre est caractérisé par un nom (souvent préfixé selon la syntaxe du SGBD) et un type de données strict, qui constitue déjà un premier niveau de contrôle de l'information.

Lors de l'appel de la procédure, il est possible de fournir les paramètres de façon positionnelle (selon leur ordre de déclaration) ou de manière nommée. L'approche nommée est fortement recommandée dès lors qu'il y a plusieurs paramètres, pour garantir la lisibilité et la robustesse du code appelant.

VALEURS PAR DÉFAUT

Certains paramètres peuvent être définis avec une valeur par défaut lors de la conception. Si l'application appelante omet de fournir ce paramètre, le système utilisera automatiquement la valeur par défaut. Cela permet de rendre certains critères optionnels (comme un seuil de recherche ou un statut par défaut).

VALIDATION ET INTÉGRITÉ DES DONNÉES

C'est une règle d'or en conception : **une procédure ne doit jamais supposer que les paramètres reçus sont corrects.** Le contrôle des entrées doit précéder toute action de modification.

- **Contrôle des valeurs :** S'assurer qu'une quantité est strictement positive, qu'un montant n'est pas nul, ou qu'une chaîne n'est pas vide.
- **Contrôle d'existence :** Vérifier qu'un identifiant client ou produit fourni en paramètre correspond bien à un enregistrement réel dans les tables.
- **Contrôles métiers :** Vérifier la cohérence d'un enchaînement (ex: s'assurer qu'une date de fin est bien postérieure à la date de début).

Si une règle n'est pas respectée, le traitement doit être immédiatement interrompu et une erreur explicite doit être levée pour informer l'appelant de l'anomalie.

Renvoyer des Résultats

Une procédure ne se contente pas toujours de modifier des données ; elle doit souvent communiquer le résultat de ses actions. Il existe trois mécanismes distincts :

Mécanisme	Rôle et Cas d'usage
Jeu de résultats (Requête directe)	Utilisé pour retourner des données tabulaires (listes, tableaux, synthèses). C'est le comportement par défaut d'une interrogation classique de données encapsulée.
Paramètre de sortie	Permet de renvoyer une ou plusieurs valeurs scalaires spécifiques (comme un identifiant nouvellement généré ou un total calculé) qui pourront être récupérées directement dans des variables par l'appelant.
Code de retour	Renvoie exclusivement un nombre entier caractérisant l'état d'exécution du traitement (souvent 0 pour le succès, et d'autres valeurs pour divers types d'erreurs). Ce mécanisme n'est pas fait pour retourner des données métiers.

Procédures métier et Transactions

La véritable puissance d'une procédure stockée se révèle lorsqu'elle encapsule une opération métier complète (ex: validation de commande = modification statut + décrémentation stock + historique).

DIFFÉRENCE ENTRE PROCÉDURE ET TRANSACTION

Il est crucial de ne pas confondre les deux concepts :

- La **procédure stockée** encapsule et réutilise un traitement. Elle n'est pas intrinsèquement transactionnelle.
- La **transaction** garantit l'atomicité et la cohérence. Elle s'assure que l'ensemble des modifications réussit ou échoue d'un seul bloc.

BONNES PRATIQUES GLOBALES

Pour construire un traitement robuste, la procédure doit associer ces concepts : recevoir les paramètres, valider les règles métiers, initier une transaction, réaliser les modifications, et intégrer un bloc de gestion des erreurs global capable d'annuler toutes les actions en cas de défaillance imprévue.